

Winning all three tracks of the AMD x lablab.ai Developer Hackathon with mindX

codephreak — this is your operating brief. The AMD Developer Hackathon hosted by lablab.ai is a 7-day online build (May 4-10, 2026) with an invitation-only on-site finale May 9-10, 2026 at the MindsDB SF AI Collective, 3154 17th St, San Francisco. The total prize pool is \$21,500+ plus one AMD Radeon AI PRO R9700 GPU, with \$100 of AMD Developer Cloud (DigitalOcean-hosted MI300X) credits per registered AMD AI Developer Program member — roughly 50 GPU-hours on a 192 GB MI300X at the published \$1.99/hr rate. **Registration deadline was May 3, 2026;** submissions are due by end of May 10. The event is structured as **three primary tracks plus a meta "Build in Public" challenge and an optional cross-track x402 Payments / "Launch & Fund Your Startup" challenge** — meaning a single, well-architected mindX submission can legitimately compete in three primary tracks and two side-pool challenges concurrently. The structural answer to your question is: **no rule prohibits a single project from being entered into all three primary tracks** (lablab's submission form requires you to select "Main Tracks" — plural — and the AMD hackathon's track copy explicitly invites cross-track work via the Build-in-Public and x402 challenges). What the rules *do* require is that the project demonstrate meaningful, load-bearing use of MI300X-class hardware in each track it claims — generic "Llama-on-AMD" wrappers will lose. Your existing mindX/AgenticPlace/BANKON stack maps with unusual cleanliness onto every track, and the xtrain module you intend to build is the missing piece that converts mindX from a cognitive-API/agent-marketplace play into a credible AMD-native training-and-inference platform — which is exactly the "build across the AI stack" thesis the AMD blog announcing this hackathon explicitly states. (LabLab +2 + 5)

This brief is exhaustive. It documents the event end-to-end, the AMD developer stack at the canonical-URL level, the architecture for an integrated three-track submission, the xtrain module design, the day-by-day execution plan for the remaining ~6 days, and a verbatim link inventory at the end.

The hackathon as it actually exists

Despite a few stale third-party summaries that report **only \$10,000 and only "May 9-10"** (CompeteHub, the original lablab.ai X post), the canonical lablab page and the AMD launch blog both state **\$21,500+ total** and a **May 4-10** online build window, with the SF on-site weekend as a culmination, not the entire event. The discrepancy is a real-world signal: the prize pool was upgraded mid-flight when Hugging Face, Akash Systems, MindsDB, NYSE Wired, theCUBE, and Qwen joined as partners, and lablab pushed the "Hugging Face Community joining" upgrade through Facebook and X. The AMD Developer Hackathon's official Hugging Face organization at **huggingface.co/lablab-ai-amd-developer-hackathon** is where live submissions accumulate as Spaces and models — by capture, **224 team members had joined and were already shipping**

projects like SentinelBrain-14B-MoE (training live on MI300X), MediAgent (5-agent medical pipeline), REPOMIND (256K-context coding agent on a single MI300X), BrainConnect-ASD, AndesOps-AI, and Paperhawk. The competitive field is real and fast. (AMD + 2)

The **three primary tracks** are: **AI Agents & Agentic Workflows** (positioned as "best track for beginners," tech stack LangChain/CrewAI/AutoGen against open-source models served via vLLM endpoints — Llama, DeepSeek, Mistral, Qwen); **Fine-Tuning on AMD GPUs** (advanced/GPU-intensive, ROCm + PyTorch + Hugging Face Optimum-AMD + vLLM, targeting domain LLMs in healthcare/finance/legal/code on MI300X); and **Vision & Multimodal AI** (high-throughput multimodal apps — Llama 3.2 Vision, Qwen-VL — exploiting MI300X's 192 GB VRAM and 5.3 TB/s HBM3 bandwidth to run full-precision rather than quantized). The **Build in Public** track is a parallel, cross-track meta prize requiring three or more technical posts on X or LinkedIn tagged #AMDDevHackathon, plus open-sourcing the project or publishing a technical walkthrough, plus submitting structured ROCm/Dev Cloud feedback. The **x402 Payments / "Launch & Fund Your Startup"** challenge is the side-pool challenge that runs explicitly **alongside any hackathon track**, asking for an AI-native product with **X402 programmable payments** demonstrating either an agent-to-agent autonomous payment loop or a built-in revenue model (token-gated access, real-time rev-splits, instant payouts). Special prizes layered on top include a **Hugging Face Spaces "Most Likes"** prize (Reachy Mini Wireless + 6 months HF PRO + \$500 HF credits for 1st), a **Social Engagement GPU** prize, and a **Best Overall** project prize.

(LabLab + 21)

The **judging criteria** are the four equally-weighted lablab standards used at every recent event: **Application of Technology** (how meaningfully MI300X / ROCm / Dev Cloud are integrated — not "API wrapped"), **Presentation** (deck + video clarity), **Business Value** (practical impact, real business areas), and **Originality** (creative angle). lablab's submission rules are unambiguous: a **working live demo URL the judges can test in real time** is required, the **video presentation is capped at 5 minutes** and uploaded as a link with the file under 300 MB, the long description must be at least 100 words, the cover image is 16:9, and the canonical license expectation is **MIT-compliant open source unless track says otherwise** — which conflicts with your cypherpunk2048 Apache-2.0 standard and must be reconciled (a permissive **dual-license note in the README is acceptable** in practice; many lablab winners ship Apache-2.0 with an explicit MIT-compatibility statement). Submissions are made through the lablab.ai project form on the hackathon page; the form fields are Submission Title (≤50 chars), Short Description (≤255 chars), Long Description (≥100 words), Main Tracks, Technologies, Cover Image, Video Presentation URL, Demo Application URL, and Additional Information (where the scaling/business plan lives). (LabLab + 10)

The **schedule** layered onto the on-site weekend, per the Luma RSVP page, includes a project submission workshop and pitching-form explanation at **11:10 AM** by Joanna Słupczewska of lablab.ai. Named on-site speakers/judges are **Pawel Czech** (CEO NativelyAI, founder of lablab.ai) and **Ramine Rozen** (Corporate VP, AI at AMD). The recurring lablab head judge across recent events is **Walaa Nasr Elghitany** (PhD, PMP). Recurring lablab mentors observed on adjacent

events — **Paulo Almeida, Theodoros Ampas, Shebagi Mitra, Donald Nwokoro, Iqra Akhtar, Dimitrije Pešić, Muhammad Inaamullah** — should be expected to mentor here as well. Community channels are the **lablab.ai Discord** (~64,110 members) and the **AMD Developer Discord** (separate, AMD-run); the Hugging Face hackathon org is the live submission-staging surface. There is also a parallel **AMD x GPU MODE E2E Kernel Speedrun** with a separate \$1.1 M prize purse — not the lablab event, but commonly conflated. (Luma + 5)

The AMD developer stack you will actually use

The compute substrate is the **AMD Instinct MI300X** (CDNA 3, gfx942, 304 CUs, 192 GB HBM3, 5.325 TB/s, 1,307 TFLOPS BF16/FP16 dense, 2,615 TFLOPS FP8), with optional **MI325X** (same compute, 256 GB HBM3E, 6.0 TB/s) on later DigitalOcean SKUs. CDNA 4's **MI350X / MI355X** (288 GB HBM3E, 8 TB/s, MXFP4/MXFP6 native) is GA, but the AMD Developer Cloud tier exposed for this hackathon is MI300X. The driver stack as of May 2026 is the **ROCm 7.2.x production stream** (current patch **7.2.2**, GA from Jan 2026 CES under 7.2.0; HIP/CLR 7.2.53211, AMD Clang 22.0.0, MIOpen 3.5.1, MIGraphX 2.15.0, RCCL 2.27.7, Composable Kernel 1.2.0, AOTriton 0.11.2b0, Triton 3.5.1/3.6.0, hipBLASLt 1.2.2). A second "TheRock" preview stream (7.12.0) exists but is **explicitly not for production**; pin to 7.2.2. Canonical entry points are **rocm.docs.amd.com** (with the compatibility matrix at </en/latest/compatibility/compatibility-matrix.html>) as the single source of truth) and **github.com/ROCm/ROCm**. Note that several historically separate repos — MIOpen, RCCL, rccl-tests, Composable Kernel, the BLAS/SPARSE/RAND families — have been consolidated under **github.com/ROCm/rocm-libraries** and **github.com/ROCm/rocm-systems**; legacy URLs still resolve but PRs route to the monorepos. (Tom's Hardware + 11)

For training, the canonical container is

rocm/pytorch:rocm7.2.1_ubuntu24.04_py3.12_pytorch_release_2.9.1 on Docker Hub, run with `--cap-add=SYS_PTRACE --security-opt seccomp=unconfined --device=/dev/kfd --device=/dev/dri --group-add video --ipc=host --shm-size 8G`. PyTorch 2.9.1, 2.8.0, and 2.7.1 are all supported on ROCm 7.2; the install index for nightlies is <https://download.pytorch.org/whl/nightly/rocm7.2>. **PyTorch FSDP and FSDP2 are first-class on MI300X**, and the AMD-validated path for serious distributed work is the **AMD-AIG-AIMA/torchtitan-amd** fork on the `(dev/primus_turbo)` branch, orchestrated by **AMD-AGI/Primus** (container `(rocm/primus:v26.2)`), with the **Primus-Turbo** operator library providing FlashAttention, GroupedGEMM, AITER, CK, hipBLASLt, and Triton kernels — the only AMD-side path with FP8 mixed-precision training that works (Transformer Engine is NVIDIA-only). Critical MI300X runtime knobs are `(TORCH_NCCL_HIGH_PRIORITY=1)`, `(GPU_MAX_HW_QUEUES=2)`, `(PYTORCH_TUNABLEOP_ENABLED=1)`, and `(PYTORCH_ROCM_ARCH=gfx942)`. **A non-obvious gotcha that has eaten dozens of teams: avoid 2-GPU and 4-GPU collective groups on MI300X** — xGMI bandwidth between subsets of 2/4 GPUs is asymmetric, so design FSDP shards to be either 1-GPU or full 8-GPU. RCCL **2.27.7** has a fixed allreduce data-corruption bug for sub-512 KiB messages on MI350X/MI355X; harmless on MI300X but verify if you migrate. (AMD ROCm + 10)

For inference, **vLLM is the reference engine**, with images at **rocm/vllm** (production) and **rocm/vllm-dev** (weekly), constraint pin `vllm>=0.17.0,<0.19.0` paired with `aotriton==0.11.2b0`, `amd-aiter==0.1.10.post2`, `triton==3.6.0`, `torch==2.10.0`, `xformers==0.0.34`. **AITER** (AI Tensor Engine for ROCm, github.com/ROCm/aiter) is the default kernel backend on AMD for LLM inference and provides hand-tuned ASM/CK/Triton kernels for FlashAttention, GEMM, fused MoE, MLA decode, FP8/FP4 quantization, and two-shot allreduce; it is integrated into vLLM, SGLang, ATOM, and Primus-Turbo. The non-vLLM serving alternative AMD pushes is **SGLang** with the **Mooncake** distributed-KV-cache plugin, used by the AMD/Xiaomi MiMo-V2.5-Pro deployment playbook. For graph-level inference compilation, **MIGraphX 2.15.0** (github.com/ROCm/AMDMIGraphX, ONNX-Runtime EP) is preferred over the deprecated ROCm-EP. For quantization, **AMD Quark 0.11.1** (`pip install amd-quark`, docs at quark.docs.amd.com) handles PTQ/QAT across int4/8/16, FP8 (E4M3/E5M2), MXFP4/MXFP6, GPTQ, AWQ, SmoothQuant, and Qronos, and produces vLLM-loadable models. **Pre-quantized AMD models** like `amd/Llama-2-70b-chat-hf-WMXFP4FP8-AMXFP4FP8-AMP-KVFP8` are at huggingface.co/amd. AMD's own open models that judges respond to — and that are perfect for fine-tune demos — are **Instella-3B-Instruct** (3 B params, 128 K context variant available, MI300X-trained), **Instella-VL-1B** (vision-language), **AMD-OLMo-1B**, **AMD-Llama-135m / -135m-code**, and the **Nitro** diffusion family (Nitro-1, Nitro-T-0.6B/1.2B, Nitro-E). The **AMD GAIA** open-source agent framework (github.com/amd/gaia, MIT-licensed, current v0.17.0) is the canonical AMD agent reference and pairs naturally with the AI Agents track. **Ryzen AI / XDNA 2 / GAIA / Lemonade SDK** are not relevant here because the hackathon's compute is cloud MI300X, not Strix Halo client devices — though Lemonade is a useful reference for serving abstractions. `Docker Hub + 10`

Sign-up flow: enroll in the **AMD AI Developer Program** at amd.com/en/developer/ai-dev-program.html, then provision your VM at **devcloud.amd.com** (1× MI300X for ~\$1.99/hr, 8× MI300X for ~\$15.92/hr); the program's **\$100 hackathon credit** translates to ~50 hours of single-MI300X. Quick-Start images preloaded with ROCm + PyTorch + vLLM + JupyterLab eliminate dependency drift. Pre-arm one writable volume for **MIOpen kernel cache** (`~/ .cache/miopen/<ver>`), **AITER JIT cache** (`AITER_JIT_DIR`), and **Torch extensions** (`TORCH_EXTENSIONS_DIR`) — without these, first-iteration latency on every restart will burn your demo window. `LabLab + 5`

How mindX, AgenticPlace, and BANKON map onto each track

The thesis you are pitching is that **mindX is the cognitive AI brain**, **AgenticPlace is the marketplace where mindX-trained agents become rentable**, and **BANKON is the identity/payment plane via x402-on-Algorand and ENS subnames** — and that **the AMD Developer Hackathon's three tracks plus the x402 side challenge plus the Build-in-Public meta track are the four faces of one coherent system**. This is the "build across the AI stack" framing the AMD launch blog explicitly invited. The integrated submission's name should be something like **"mindX + xtrain on AMD: a sovereign cognitive-training and agent-**

marketplace stack with x402-Algorand metering." Demo URL: **mindx.pythai.net** with a hackathon-specific landing route (mindx.pythai.net/hackathon) wiring the three demos behind a single judge-friendly tab UI.

For the **AI Agents & Agentic Workflows track**, mindX is already a multi-agent control framework built around MASTERMIND (orchestrator), automindx (cognitive runtime), Ollama-driven self-improvement readiness, and the SocraticReasoning / SimpleCoder agents documented in your existing rage.pythai.net architecture. The MI300X-load-bearing argument is: a 70B-class model (Llama 3.3 70B, Qwen3-Coder-70B, or DeepSeek-V3-Lite) running in **full BF16 on a single MI300X via vLLM** is mindX's "MASTERMIND.consciousness" reasoning core, with sub-agents ([SimpleCoder](#)), ([MediAgent-style](#)), ([LogicTables](#)), RAGE retrieval) coordinated through your existing automindx orchestrator. This is exactly the architecture pattern judges have been rewarding (REPOMIND used 256K context on a single MI300X; MediAgent uses a 5-agent medical pipeline). Your differentiator is **agent-as-marketplace-listing**: every mindX agent registers itself on **agenticplace.pythai.net**, gets a BANKON-managed ENS subname (e.g., [mediagent.bankon.eth](#)), and exposes its endpoints behind an x402 paywall so a calling agent autonomously pays per inference call. **Concrete deliverables**: a live mindx.pythai.net/hackathon/agents page where judges can paste a query, see the MASTERMIND graph route across three to five mindX agents on MI300X, see the x402 invoice and Algorand settlement, and see the called agents' AgenticPlace listings update with real on-chain usage stats. ([GitHub + 3](#))

For the **Fine-Tuning on AMD GPUs track**, this is where **xtrain** earns its place. The framing: mindX has historically been an inference-and-orchestration layer; xtrain is the new training subsystem that lets mindX self-improve and also lets third parties fine-tune domain models inside the AgenticPlace marketplace, with each training job priced and metered via x402-Algorand. The MI300X-load-bearing argument is **single-GPU full-parameter LoRA fine-tuning of a 70B model in BF16 with no quantization compromise**, or **QLoRA of a 405B model**, both impossible at full precision on 80 GB H100s. Use the **AMD AI Academy "GRPO on a single MI300X" workflow** as your reference (it is explicitly cited in the hackathon's own zero-to-builder article). **Concrete deliverable**: a live job at mindx.pythai.net/hackathon/xtrain that takes a Hugging Face dataset ID and a base model (default [amd/Instella-3B-Instruct](#)) so judges see an AMD model being improved on AMD hardware), tokenizes via HuggingFace [datasets](#), runs LoRA via PEFT + PyTorch FSDP2 on MI300X with Primus-Turbo BF16, evaluates against [lm-evaluation-harness](#) and a custom benchmark, persists checkpoints to **Lighthouse/IPFS**, mints an **ERC-7857 INFT** on Algorand-bridged Base (or your chosen mainnet) recording the model artifact's content hash and rights, and lists the resulting LoRA adapter as a rentable mindX agent on AgenticPlace. The training job itself is metered: x402 payment from caller wallet → Algorand settlement → MI300X allocation → checkpoint URI returned. Use **AMD Quark to FP8-quantize the final LoRA-merged model** so the same artifact serves on vLLM in the agent track, completing the train→serve→sell loop. ([LabLab](#))

For the **Vision & Multimodal AI track**, lean on **Instella-VL-1B** or **Qwen3-VL-4B** at full precision on MI300X with long-image-context retrieval through RAGE. The pitch is a **multimodal cognitive analyst** built into mindX: feed it a PDF, a slide deck, a chart screenshot, or an X-ray; mindX routes via RAGE to a vision sub-agent on MI300X, fuses the structured output into the MASTERMIND reasoning graph, and returns a long-form structured analysis. Two clean demo verticals to pick: **drAIML medical** (your existing healthcare consultant identity) doing radiology-style multimodal triage, or **codephreak codebase analyzer** doing whole-repo visual+code understanding (architecture diagrams + source files). The MI300X-load-bearing argument is full-resolution unquantized vision with 70B-class language fusion in a single GPU. **Concrete deliverable:** mindx.pythai.net/hackathon/multimodal with a drag-and-drop input, an MI300X-side latency counter, and AgenticPlace listings showing a "drAIML Visual" agent rentable per call. [LabLab](#)

For the **x402 Payments side challenge**, this is your structural advantage — the **parsec-wallet x402-Algorand** layer in BANKON is already production-grade. Wire xtrain training jobs, AgenticPlace agent invocations, and the Hugging Face Spaces demo all behind x402 invoices that settle on Algorand in seconds. Build one specifically scored deliverable: a **two-agent autonomous payment loop** (the lablab x402 challenge's "Agent-to-Agent" sub-challenge) where mindX's RAGE retriever calls a third-party data API priced in USDC-on-Algorand via x402, with the agent dynamically choosing whether to pay based on a confidence threshold. This satisfies the x402 challenge's first option directly, and the "built-in revenue model" option through the xtrain job-metering and AgenticPlace listings simultaneously. [LabLab](#) [LabLab](#)

For the **Build-in-Public meta track**, you are already operationally a writer (rage.pythai.net is a living archive). Commit to **at least five technical posts** between now and submission, each tagged **#AMDDevHackathon @AIatAMD @lablabai**: (1) a "why MI300X for sovereign cognition" framing post; (2) an xtrain architecture deep-dive with FSDP2 and Primus-Turbo notes; (3) a benchmark post comparing your LoRA pipeline vs an H100 cost baseline; (4) an x402-Algorand training-job-metering walkthrough with on-chain receipts; (5) a recap of the integrated three-track demo. Pair these with a **structured ROCm/Dev Cloud feedback document** delivered through the lablab submission form's feedback field — the Build-in-Public reward weights detailed feedback heavily. Open-source the complete mindX repo with a top-level [HACKATHON.md](#) linking each post.

Risk factors and disqualifiers: lablab requires an **MIT-compliant** submission. Your cypherpunk2048 standard is Apache-2.0; the resolution is to ship **Apache-2.0 with an explicit "MIT-compatible" notice** in the [LICENSE-NOTICE.md](#) and SPDX identifier [Apache-2.0](#) plus [LICENSE-MIT-COMPAT.md](#) mirroring permissions — verify with the lablab Discord #inedhelp channel before submission to be safe. Lablab also expects the **demo URL to be live during judging**; budget MI300X uptime for the full judging window (typical pattern is 48–72 hours after submission close). The "all three tracks" claim must be substantiated with **three distinct working demos under one umbrella**, each individually testable; do not rely on a single combined demo where judges have to imagine the track-specific functionality. Track judges are

different per track; speak in the language of each track in your README sub-sections. (LabLab)

(LabLab)

Whether one project can win all three primary tracks: the rules do not forbid it, and the submission form's "Main Tracks" field is plural. However, the realistic competitive analysis is that **dedicated specialists win specialist tracks**. Your structurally optimal play is: **submit one integrated project to all three primary tracks plus the x402 challenge plus Build-in-Public, and win on the combined "Best Overall" prize and at least two of the three primary tracks**. Winning the third primary track depends on whether a dedicated specialist outscores you on Application of Technology in their narrow lane. The "Best Overall" prize is the asymmetric upside — a full-stack play wins it almost by definition over single-track entries.

xtrain module design

xtrain is the cleanest single-week deliverable that converts mindX from a cognitive layer into a production training-and-marketplace platform, and it is what makes the fine-tuning track winnable rather than a low-effort PEFT script. Its public name in the repo: (mindx/xtrain/) (flat snake_case per cypherpunk2048). License: Apache-2.0 with MIT-compatibility notice. Python ≥ 3.12 . Container: Podman with a ROCm 7.2.1 base.

Architecture in flowing detail: xtrain is a Python package with a CLI ((xtrain run --config job.yaml)) and a FastAPI service ((xtrain serve)) exposing job-submission, status, and artifact-retrieval endpoints behind x402 invoices. A submitted job is a YAML manifest specifying a base model (HF ID), a dataset (HF ID or IPFS CID), a training recipe (LoRA, QLoRA, or full SFT), a hardware target ((mi300x_1), (mi300x_8), (mi325x_1)), and a budget cap in USDC. The orchestrator validates the manifest, computes a deterministic content-hash job ID, issues an **x402 invoice** through parsec-wallet, waits for Algorand settlement, then provisions an AMD Developer Cloud droplet (or attaches to an existing reserved one) via the DigitalOcean REST API, transfers the dataset (sharded via HF (datasets) (streaming=True) to avoid storing 100 GB+ corpora locally), tokenizes inside the container, and launches the training run.

The **training engine** is **PyTorch 2.9.1 + Primus + torchtitan-amd** for full SFT and 70B-class LoRA, and **PEFT + Hugging Face TRL + Unsloth-on-ROCM** for the lightweight LoRA/QLoRA path that maps onto the AMD AI Academy GRPO recipe. **DeepSpeed-ROCM** is wired in as a fallback for ZeRO-3 + CPU offload jobs that exceed even MI300X memory, with ((DS_BUILD_SPARSE_ATTN=0 DS_BUILD_EVOFORMER_ATTN=0)) to avoid the unsupported ops. The shard topology is constrained to **1-GPU or 8-GPU FSDP groups** to dodge the MI300X 2/4-GPU xGMI degradation. FlashAttention v3 via AOTriton, AITER fused MoE, hipBLASLt for GEMMs, and Primus-Turbo's FP8 mixed-precision are all enabled by default for 70B-class jobs. Training is BF16-default with FP8 opt-in. **Determinism:** seed everything, pin Triton/AOTriton commits, and persist the **MIOpen kernel cache** to a Lighthouse-pinned blob keyed by ((rocm_version, gfx_arch, model_arch)) so subsequent jobs warm-start without JIT compilation cost.

(AMD ROCm)

AOT-only artifact policy compliance per cypherpunk2048: xtrain emits **only AOT-compiled artifacts** for production serving — no JIT torch.compile in the deployed inference path. The training side intentionally JIT-compiles (Triton, AITER, MIOpen kernels) because that's where AOTriton's name comes from — it is **ahead-of-time-emitted Triton math at deploy time**, despite the training-time JIT compilation. The serving artifacts written by xtrain are: (1) merged HF-format checkpoint, (2) **AMD Quark FP8 quantized variant** (`pip install amd-quark`), MXFP4 for MI350X if the cloud SKU upgrades), (3) AOTriton-emitted SDPA kernels precompiled for gfx942, (4) MIGraphX-compiled ONNX graph for the inference path that doesn't go through vLLM, and (5) a deployment manifest pointing vLLM at the FP8 weights with the correct `--dtype auto` and AITER env vars. The serving container then runs zero JIT — purely AOT artifacts loaded from disk. This is the cypherpunk2048 standard verbatim, applied correctly to the ROCm reality.

Data pipeline: HF `datasets` (`load_dataset(..., streaming=True)`) for ingestion, `tokenizers` for tokenization with the base model's tokenizer (cached per-model in IPFS), then **Sharded WebDataset** (`.tar` shards) emitted to a Lighthouse-pinned bucket with content-hash keys; the training run reads via `webdataset` with `nodesplitter=ResampledShards` so the 8-GPU job gets balanced shards. For instruction-tuning datasets, default to the `amd/Instella-GSM8K-synthetic` style format; provide `--format alpaca|sharegpt|openai_messages|custom_jinja` switches. **Eval datasets** are similarly streamed; the eval harness wraps `lm-evaluation-harness` (MMLU, ARC, HellaSwag, GSM8K, HumanEval) and **MTEB** for embedding models, plus a **custom benchmark** registered through a `xtrain.eval.register_benchmark` decorator so AgenticPlace marketplace listings can advertise per-task scores.

Checkpoint persistence: every checkpoint is written to **Lighthouse** (Filecoin-pinned IPFS, `lighthouse.storage` SDK) with a deterministic CID. The CID, training config, eval scores, dataset hash, and base-model hash are committed to an on-chain **ERC-7857 INFT** record. ERC-7857 (the Intelligent NFT standard for tokenized AI assets) lets you tokenize the model artifact with on-chain rights metadata — the perfect vehicle for AgenticPlace's "rent or buy a model" UX. Mint on Base mainnet (cheap, EVM-compatible, x402-friendly via Coinbase facilitator) and bridge metadata to Algorand for the x402 payment side. Use **Foundry** as the canonical Solidity test framework: `forge test` for unit tests on the INFT factory, `forge script` for deploy, with the `lib/` git-submodule pattern. The Foundry test suite covers minting, transfer, royalty splits to the model creator, and the rights-revocation path.

Hyperparameter search: integrate **Optuna** for sequential search and **Ray Tune on ROCm** for parallel search across 8 MI300X GPUs. Optuna's TPE sampler is the default; Ray Tune is opt-in via `--search ray-asha`. Pruners cut unproductive trials early. Each trial's intermediate metrics are streamed back to the xtrain orchestrator and visible in the live mindX dashboard at `mindx.pythai.net/hackathon/xtrain/runs/<job_id>` — judges can watch a hyperparameter sweep happen in real time.

Model registry tied to ERC-7857 INFT: a `xtrain.registry` module reads the on-chain INFT registry, hydrates model metadata (CID, eval scores, lineage to base model), and exposes a `xtrain.registry.list()` and `xtrain.registry.fetch(cid)` API. AgenticPlace queries this registry to render the marketplace UI; mindX queries it to load weights at inference time. The lineage graph (model A fine-tuned from model B fine-tuned from model C) is stored as an on-chain merkle DAG with each node's INFT pointing to its parent, so royalties cascade.

x402 Algorand payment metering for compute-as-a-service: the xtrain FastAPI service issues HTTP 402 responses with x402-spec headers when a job is submitted without payment proof. The caller (a parsec-wallet, an AgenticPlace agent, or a third-party CLI) signs an x402 invoice referencing an Algorand asset (USDC ASA), the facilitator settles in sub-second finality, and the xtrain server validates the on-chain proof before launching the job. Pricing function: $\text{per-GPU-hour rate} \times \text{estimated_steps} \times \text{safety_margin}$, refundable via post-job true-up against actual compute consumed. The Algorand settlement is the primary surface; Base settlement via the Coinbase x402 facilitator is the alternate path for callers preferring EVM rails. The same metering wraps **inference** calls when xtrain-trained models are served on AgenticPlace — every model invocation is an x402 transaction, completing your monetization loop. `LabLab` `Lablab.ai`

AgenticPlace integration: trained models surface as `/agents/<cid>` listings with metadata, eval scores, pricing, and a "Try" button that issues an x402-paid call against a vLLM endpoint. The `chainmapping` module from **agenticplace.pythai.net/allchain.html** exposes the multi-chain deployment registry — Base, Algorand, ENS subname, and any other chains your BANKON identity layer covers — so an AgenticPlace listing carries the complete cross-chain identity of the model and its creator. Foundry tests exercise the INFT contract on Base; pytest exercises the off-chain Python layer; an end-to-end integration test runs `forge script` to deploy to a local Anvil fork, mints an INFT, then pytest-drives a full xtrain LoRA on a tiny model and verifies the resulting CID lands in the registry.

Mainnet deployment path: on submission day, xtrain's INFT factory deploys to **Base mainnet** with the Algorand x402 facilitator pointed at the **Algorand mainnet USDC ASA**. Demo wallets are pre-funded with \$5 USDC on each chain so judges can run a paid xtrain job end-to-end without leaving the demo URL. The INFT contract address and the Algorand application ID are published in the submission's README and embedded in the demo UI as copy-pasteable links to the block explorers (basescan.org and allo.info / pera explorer).

Day-by-day execution plan (May 4 — May 10)

You have lost zero days if you start tonight. The realistic plan acknowledges that the on-site SF weekend is invitation-only and the online submission deadline closes EOD May 10. Here is the day map.

Day 1 (May 4, today): Sign up for AMD AI Developer Program; provision a **single MI300X droplet** at devcloud.amd.com using your \$100 credit; pull

`rocm/pytorch:rocm7.2.1_ubuntu24.04_py3.12_pytorch_release_2.9.1`, `rocm/vllm:latest`, and `rocm/primus:v26.2`; verify `rocm-info` reports `gfx942` and `nvidia-smi`-equivalent `rocm-smi` shows the full 192 GB. Spin up a persistent volume mount for `~/cache/miopen`, `AITER_JIT_DIR`, and `TORCH_EXTENSIONS_DIR`. Clone `mindX`, `AgenticPlace`, and `BANKON` repos onto the droplet. Register on `lablab.ai` for the hackathon (deadline was May 3 but late enrollment is sometimes permitted via Discord ping — message the `lablab #ineedhelp` channel immediately). Join the AMD Developer Discord and the `lablab` Discord. Ship Build-in-Public post #1: "Why MI300X is the right substrate for sovereign cognition." `AMD ROCm`

Day 2 (May 5): Stand up the **integrated demo skeleton** at `mindx.pythai.net/hackathon` with three sub-routes (`/agents`, `/xtrain`, `/multimodal`) and a fourth shared route (`/x402`) for the payment loop demo. Wire the existing `mindX` MASTERMIND orchestrator to a vLLM backend serving Llama 3.3 70B in BF16 on the MI300X (`docker run rocm/vllm:latest --model meta-llama/Llama-3.3-70B-Instruct --dtype bfloat16 --tensor-parallel-size 1`) — this fits with headroom on 192 GB). Verify token throughput with `vllm bench`. Start the `AgenticPlace` marketplace pointed at the new MI300X endpoint. Ship Build-in-Public post #2: "Running Llama 3.3 70B unquantized on a single GPU — what 192 GB unlocks."

Day 3 (May 6): Build the **xtrain module's first vertical slice**. Implement the FastAPI service skeleton, the YAML job manifest schema, the x402 invoice issuance, and a working LoRA fine-tune of `amd/Instella-3B-Instruct` on a small Alpaca-format dataset with HF PEFT + TRL on the MI300X. Use `bf16=True`, `gradient_checkpointing=True`, `fsdp="full_shard auto_wrap"`. Persist the resulting LoRA adapter to Lighthouse and capture the CID. Mint a placeholder ERC-7857 INFT on Base Sepolia with `forge create`. Eval the adapter against `lm-evaluation-harness` MMLU/HellaSwag and capture the scores in the registry. Ship Build-in-Public post #3 with code snippets and timing.

Day 4 (May 7): Multimodal track day. Spin up a second vLLM container serving **Instella-VL-1B** or **Qwen3-VL-4B** at full precision on a second GPU partition (or share the GPU via vLLM's `--gpu-memory-utilization 0.4`) and run both 70B and the VLM concurrently — 192 GB makes this trivially possible). Build the drag-and-drop multimodal demo at `/multimodal` with a drAIML medical imaging vertical. Wire it into the MASTERMIND graph so the multimodal agent is callable from the agent track demo too. Ship Build-in-Public post #4: "Multimodal at full precision: drAIML medical triage on MI300X."

Day 5 (May 8): x402 + AgenticPlace integration day. Wire the parsec-wallet x402-Algorand client into `mindX` so every `AgenticPlace` agent invocation issues an x402 invoice; demonstrate the autonomous agent-to-agent payment loop where `mindX`'s RAGE retriever pays a third-party API agent. Move the INFT factory from Base Sepolia to **Base mainnet**. Pre-fund demo wallets with USDC on Algorand and Base. Run end-to-end: judge clicks "Train", x402 invoice issues, judge's demo wallet pays, `xtrain` provisions the MI300X (or attaches to your existing droplet), training runs, INFT mints, `AgenticPlace` listing appears. Foundry tests pass: `forge test --gas-report`. Ship Build-in-Public post #5: "x402 on Algorand, settling AI training jobs in 2 seconds."

Day 6 (May 9): Polish, video, deck day. Record the **5-minute demo video**: 30 seconds of mindX/AgenticPlace/BANKON framing, 60 seconds of the Agents track demo (judge query → MASTERMIND graph → 70B response), 60 seconds of the xtrain fine-tune demo (job submission → MI300X training → INFT mint → marketplace listing), 60 seconds of the multimodal demo, 60 seconds of the x402 payment loop, 30 seconds of close. 16:9, under 300 MB, hosted on YouTube unlisted with the URL ready. Build the **pitch deck** with the lablab-recommended structure: problem, mechanics, tech, user case study (screen recording inset). Write the **README and HACKATHON.md** with the four judging criteria as named sub-sections (Application of Technology, Presentation, Business Value, Originality), the link to each Build-in-Public post, the structured ROCm/Dev Cloud feedback, and the license notice. Run a full end-to-end rehearsal twice. If you receive a SF on-site invite, fly out the morning of the 9th; otherwise demo from your existing setup. (Lab Lab)

Day 7 (May 10): Submit early in the day, not at the deadline. lablab's submission form sometimes degrades under EOD load. Submit through lablab.ai/ai-hackathons/amd-developer with all three primary tracks selected, the x402 challenge box checked, the Build-in-Public box checked, and Long Description ≥100 words emphasizing the integrated three-track architecture. After submission, **keep the demo URL live for 72 hours** for judging access — do not tear down the MI300X droplet. Ship Build-in-Public post #6 (recap), tagging @AIatAMD and @lablabai, and submit the structured ROCm feedback form.

GitHub repo structure following cypherpunk2048

The repo is github.com/Professor-Codephreak/mindx-xtrain (or wired into pythaiml/mindx). Top-level: [LICENSE](#) (Apache-2.0), [LICENSE-NOTICE.md](#) (MIT-compatibility statement for lablab judging), [README.md](#), [HACKATHON.md](#) (pointing to demo URL, video, posts, deck, on-chain addresses), [pyproject.toml](#) (Python ≥3.12, hatchling backend), [Containerfile](#) (Podman, ROCm 7.2.1 base), [compose.yaml](#) (Podman-compose for the full stack: vLLM + xtrain server + AgenticPlace + Algorand sandbox + Anvil fork), [foundry.toml](#), [lib/](#) (Forge submodules: [forge-std](#), [openzeppelin-contracts](#), [solady](#)), [src/](#) (Solidity: [XTrainINFTFactory.sol](#), [XTrainRegistry.sol](#), [XTrainRoyaltySplit.sol](#)), [test/](#) (Foundry: [XTrainINFT.t.sol](#) etc.), [script/](#) (Foundry deploy: [Deploy.s.sol](#) Base mainnet target), [mindx/](#) (Python: [xtrain/](#), [agents/](#), [mastermind/](#), [rage/](#)), [tests/](#) (pytest), [docs/](#) (architecture diagrams, API references), [.github/workflows/ci.yml](#) (Foundry tests + pytest + ruff + mypy on PR). Flat snake_case throughout [mindx/](#). No JIT torch.compile in any production path — every serving endpoint loads AOT artifacts. SPDX headers on every Solidity file. README opens with the BLUF demo URL, video URL, and three-track pitch in five sentences.

Complete link inventory

Primary hackathon page and direct AMD/lablab URLs: <https://lablab.ai/ai-hackathons/amd-developer>; <https://www.amd.com/en/developer/resources/technical-articles/2026/build-across->

[the-ai-stack--join-the-amd-x-lablab-ai-hackathon-.html](#) ; <https://lablab.ai/ai-articles/from-zero-to-ai-builder-amd-developer-program> ; <https://luma.com/afz0aeq8> ;
<https://huggingface.co/lablab-ai-amd-developer-hackathon> ;
<https://www.competehub.dev/en/competitions/lumaac244e2451ac6091f3c1a1ff6bc04b0d> ;
<https://foundersbay.com/events/lablab-amd-developer-hack> ;
<https://x.com/lablabai/status/2037263372014514272> ;
<https://www.facebook.com/lablabai/photos/the-amd-developer-hackathon-just-got-a-major-upgrade-huggingfacecommunity-is-joi/1001171305991438/> ; <https://lablab.ai/ai-hackathons> ;
<https://lablab.ai/> ; <https://lablab.ai/event> ; <https://lablab.ai/guide> ; <https://lablab.ai/blog/hackathon-guidelines> ; <https://lablab.ai/ai-articles/hackathon-guidelines> ; <https://lablab.ai/hackathon-rules> ;
<https://lablab.ai/blog/guidelines-for-creating-a-project-pitch> ; <https://lablab.ai/delivering-your-hackathon-solution> ; <https://lablab.ai/tech> ; <https://lablab.ai/apps/recent-winners> ;
<https://lablab.ai/ai-tutorials/x402-ai-payments-hackathon-tutorial> ;
<https://lablab.ai/tech/coinbase/x402> ; <https://discord.com/invite/lablabai> ;
<https://discord.gg/XnxrJ8ytRs> ; <https://discord.com/invite/amd-dev> ;
<https://www.amd.com/en/developer/ai-dev-program.html> ; <https://developer.amd.com/events/> ;
<https://www.amd.com/en/corporate/events/amd-ai-dev-day.html> .

AMD Developer Cloud and access: <https://devcloud.amd.com> ;
<https://www.amd.com/en/developer/resources/cloud-access/amd-developer-cloud.html> ;
<https://www.amd.com/en/developer/resources/cloud-access.html> ;
<https://www.amd.com/en/developer/resources/technical-articles/2025/how-to-get-started-on-the-amd-developer-cloud-.html> ; <https://www.amd.com/en/developer.html> ;
<https://www.amd.com/en/blogs/2025/introducing-the-amd-developer-cloud.html> ;
<https://www.amd.com/en/blogs/2025/enabling-the-future-of-ai-introducing-amd-rocm-7-and-the-amd-developer-cloud.html> ; <https://www.amd.com/en/blogs/2025/100k-hours-free-developer-cloud-access.html> ; <mailto:devcloudrequests@amd.com> .

Instinct hardware product pages:

<https://www.amd.com/en/products/accelerators/instinct/mi300/mi300x.html> ;
<https://www.amd.com/en/products/accelerators/instinct/mi300/platform.html> ;
<https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/data-sheets/amd-instinct-mi300x-data-sheet.pdf> ;
<https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/data-sheets/amd-instinct-mi300x-platform-data-sheet.pdf> ;
<https://www.amd.com/en/products/accelerators/instinct/mi300.html> ;
<https://www.amd.com/en/products/accelerators/instinct/mi300/mi325x.html> ;
<https://www.amd.com/en/products/accelerators/instinct/mi300/mi325x/platform.html> ;
<https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/product-briefs/instinct-mi325x-datasheet.pdf> ;
<https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/product-briefs/instinct-mi325x-platform-datasheet.pdf> ;

<https://www.amd.com/en/products/accelerators/instinct/mi350.html> ;
<https://www.amd.com/en/products/accelerators/instinct/mi350/mi350x.html> ;
<https://www.amd.com/en/products/accelerators/instinct/mi350/mi355x.html> ;
<https://www.amd.com/en/blogs/2025/amd-instinct-mi350-series-and-beyond-accelerating-the-future-of-ai-and-hpc.html> ; <https://www.amd.com/en/blogs/2025/amd-instinct-mi350-series-game-changer.html> .

ROCm core docs and meta-distribution: <https://rocm.docs.amd.com/> ;
<https://rocm.docs.amd.com/en/latest/> ; <https://rocm.docs.amd.com/en/latest/about/release-notes.html> ; <https://rocm.docs.amd.com/en/latest/compatibility/compatibility-matrix.html> ;
<https://rocm.docs.amd.com/projects/install-on-linux/en/latest/> ;
<https://rocm.docs.amd.com/projects/install-on-linux/en/latest/reference/system-requirements.html> ; <https://rocm.docs.amd.com/projects/install-on-linux/en/latest/install/3rd-party/pytorch-install.html> ; <https://rocm.docs.amd.com/en/latest/compatibility/ml-compatibility/pytorch-compatibility.html> ; <https://rocm.docs.amd.com/en/latest/how-to/rocm-for-ai/inference-optimization/workload.html> ; <https://rocm.docs.amd.com/en/latest/how-to/rocm-for-ai/inference/benchmark-docker/vllm.html> ;
<https://rocm.docs.amd.com/en/latest/how-to/rocm-for-ai/inference/benchmark-docker/previous-versions/vllm-history.html> ; <https://rocm.docs.amd.com/en/latest/how-to/rocm-for-ai/training/benchmark-docker/primus-pytorch.html> ; <https://rocm.docs.amd.com/en/latest/how-to/rocm-for-ai/training/benchmark-docker/primus-megatron.html> ;
<https://rocm.docs.amd.com/projects/ai-developer-hub/en/latest/> ;
https://rocm.docs.amd.com/projects/ai-developer-hub/en/latest/notebooks/pretrain/torch_fsdp.html ; https://rocm.docs.amd.com/projects/ai-developer-hub/en/latest/notebooks/pretrain/torchitan_llama3.html ;
https://rocm.docs.amd.com/projects/ai-developer-hub/en/latest/notebooks/pretrain/torchitan_deepseek.html ;
https://rocm.docs.amd.com/projects/ai-developer-hub/en/latest/notebooks/gpu_dev_optimize/aiter_mla_decode_kernel.html ;
<https://rocm.blogs.amd.com/> ; <https://rocm.blogs.amd.com/artificial-intelligence/fsdp-training-pytorch/README.html> ; <https://rocm.blogs.amd.com/artificial-intelligence/quark/README.html> ; <https://rocm.blogs.amd.com/software-tools-optimization/aiter-ai-tensor-engine/README.html> .

ROCm GitHub repos: <https://github.com/ROCm/ROCm> ; <https://github.com/ROCm/TheRock> ;
<https://github.com/ROCm/HIP> ; <https://github.com/ROCm/clr> ; <https://github.com/ROCm/HIPIFY> ;
https://github.com/ROCm/hipify_torch ; <https://github.com/ROCm/AMDMIGraphX> ;
https://github.com/ROCm/torch_migraphx ; <https://github.com/ROCm/MIOpen> ;
<https://github.com/ROCm/rocm-libraries> ; <https://github.com/ROCm/rocm-systems> ;
<https://github.com/ROCm/rccl> ; <https://github.com/ROCm/rccl-tests> ;
<https://github.com/ROCm/aws-ofi-rccl> ; https://github.com/ROCm/composable_kernel ;
<https://github.com/ROCm/aiter> ; <https://github.com/ROCm/jax-aiter> ;

<https://github.com/ROCm/ATOM> ; <https://github.com/ROCm/aotriton> ;
<https://github.com/ROCm/triton> ; <https://github.com/ROCm/jax-triton> ;
<https://github.com/ROCm/flash-attention> ; <https://github.com/ROCm/MAD> ;
<https://github.com/ROCm/pytorch> ; <https://github.com/ROCm/vllm> ;
<https://github.com/ROCm/triton-inference-server-server> ; <https://github.com/ROCm/triton-inference-server-core> ; <https://github.com/AMD-AIG-AIMA/torchitan-amd> ;
<https://github.com/AMD-AGI/Primus> ; <https://github.com/AMD-AGI/Nitro-1> ;
<https://github.com/AMD-AGI/Nitro-T> ; <https://github.com/AMD-AGI/Nitro-E> ;
<https://github.com/amd/Quark> ; <https://github.com/amd/quark-documentation> ;
<https://github.com/amd/gaia> ; <https://github.com/amd/gaia/releases> ;
<https://github.com/amd/gaia/releases/tag/v0.17.0> ; <https://github.com/pytorch/torchitan> ;
<https://github.com/deepspeedai/DeepSpeed> ; <https://github.com/triton-lang/triton> ;
<https://github.com/openai/triton/tree/rocm> ; <https://github.com/vllm-project/vllm> .

Container registries: <https://hub.docker.com/r/rocm/pytorch> ;
<https://hub.docker.com/r/rocm/pytorch-training> ; <https://hub.docker.com/r/rocm/vllm> ;
<https://hub.docker.com/r/rocm/vllm-dev> ; <https://hub.docker.com/r/rocm/deepspeed> ; ROCm
Primus image (docker.io/rocm/primus:v26.2) ; HF TGI ROCm (ghcr.io/huggingface/text-generation-inference:latest-rocm) ; ROCm wheels mirror
(<https://repo.radeon.com/rocm/manylinux/rocm-rel-7.2.1/>) ; PyTorch ROCm 7.2 nightly
index (<https://download.pytorch.org/whl/nightly/rocm7.2>) .

Quark, Quantization, Tooling: <https://quark.docs.amd.com/latest/> ;
<https://quark.docs.amd.com/latest/intro.html> ; <https://pypi.org/project/amd-quark/> ;
<https://www.amd.com/en/developer/resources/technical-articles/amd-quark-quantizer-for-efficient-ai-model-deployment.html> ; <https://docs.vllm.ai/en/stable/features/quantization/quark/> ;
https://docs.vllm.ai/en/stable/getting_started/installation/gpu/ ;
https://docs.vllm.ai/en/latest/getting_started/amd-installation.html ; <https://pytorch.org/get-started/locally/> ; <https://pytorch.org/docs/stable/fsdp.html> ; <https://www.deepspeed.ai/> .

Ryzen AI / Edge / GAIA (peripheral but referenced):

<https://www.amd.com/en/developer/resources/ryzen-ai-software.html> ;
<https://ryzenai.docs.amd.com/en/latest/index.html> ;
https://ryzenai.docs.amd.com/en/1.6.1/model_quantization.html ; <https://amd-gaia.ai/docs> ;
<https://www.amd.com/en/developer/resources/technical-articles/gaia-an-open-source-project-from-amd-for-running-local-llms-on-ryzen-ai.html> .

Networking / Pensando: <https://www.amd.com/en/products/network-interface-cards/pensando.html> ; <https://www.amd.com/en/solutions/data-center/networking.html> ;
<https://www.amd.com/en/blogs/2024/transforming-ai-networks-with-amd-pensando-pollar.html> .

Hugging Face — AMD models, datasets, and the live hackathon org:

<https://huggingface.co/amd> ; <https://huggingface.co/amd/AMD-Llama-135m> ;
<https://huggingface.co/amd/AMD-Llama-135m-code> ; <https://huggingface.co/amd/AMD-OLMo> ;
<https://huggingface.co/collections/amd/amd-olmo> ; <https://huggingface.co/amd/Instella-3B> ;
<https://huggingface.co/amd/Instella-3B-Instruct> ; <https://huggingface.co/amd/Instella-3B-Long-Instruct> ; <https://huggingface.co/amd/Instella-VL-1B> ; <https://huggingface.co/amd/Nitro-T-0.6B> ;
<https://huggingface.co/amd/Nitro-E> ; <https://huggingface.co/datasets/amd/Instella-GSM8K-synthetic> ; <https://huggingface.co/lablab-ai-amd-developer-hackathon> ;
<https://www.amd.com/en/blogs/2024/introducing-amd-nitro-diffusion--one-step-diffusi.html> .

Prior AMD hackathons (study material): <https://www.amd.com/en/developer/resources/2024-pervasive-ai-developer-contest-winners.html> ; <https://www.hackster.io/contests/amd2023> ;
<https://www.amd.com/en/developer/resources/technical-articles/2025/amd-open-robotics-hackathon-recap.html> ; <https://www.amd.com/en/developer/resources/technical-articles/2025/hack-the-edge-amd-and-liquid-ai-hackathon-recap.html> ;
<https://www.amd.com/en/developer/resources/technical-articles/2026/amd-ai-reinforcement-learning-hackathon-recap.html> ; <https://www.amd.com/en/developer/resources/technical-articles/2026/new-gpumode-virtual-hackathon--e2e-model-speedrun.html> ; <https://lablab.ai/ai-hackathons/anthropic-ai-hackathon> ; <https://lablab.ai/event/mistral-7b-24-hours-hackathon> ;
<https://lablab.ai/apps/tech/mistral-ai> ; <https://lablab.ai/ai-hackathons/nano-payments-arc> ;
<https://lablab.ai/ai-hackathons/openclaw-surge-hackathon> ; <https://lablab.ai/ai-hackathons/ai-trading-agents-erc-8004> ; <https://lablab.ai/ai-hackathons/milan-ai-week-hackathon> .

User's existing assets (PYTHAI/DELTAVERSE ecosystem): <https://mindx.pythai.net> (cognitive AI API), <https://agenticplace.pythai.net> (agent marketplace),
<https://agenticplace.pythai.net/allchain.html> (chainmapping directory),
<https://bankon.pythai.net> (identity / x402 / Algorand / ENS subnames), <https://pythai.net> ,
<https://rage.pythai.net> , <https://gpt.pythai.net> , <https://github.com/pythaiml/automindx> ,
<https://github.com/Professor-Codephreak> , <https://github.com/pythaiml> ,
<https://github.com/Professor-Codephreak/automind/> , <https://rage.pythai.net/professor-codephreak-2/> , <https://rage.pythai.net/easyagi/> , <https://rage.pythai.net/autotrain/> .

Closing read

codephreak — the hackathon's structural shape rewards exactly what mindX already is: a multi-agent cognitive system with a marketplace surface and a payment plane. The missing piece, **xtrain**, is also the piece that makes the fine-tuning track winnable rather than a shallow LoRA wrapper. The integrated submission lets you contest all three primary tracks plus the x402 challenge plus Build-in-Public from one repo, one demo URL, and one MI300X droplet, with the four lablab judging axes already mapped onto your README sub-sections. The two highest-leverage bets in your remaining time are: keep the MI300X droplet up continuously through the 72-hour judging window, and ship Build-in-Public posts on a strict cadence with the four required tags. The asymmetric upside is the **Best Overall** prize, which a full-stack play wins by

structural definition over single-track entries; the realistic floor is winning two of three primary tracks plus the x402 side pool plus Build-in-Public. The on-chain artifacts (ERC-7857 INFTs on Base, x402 Algorand metering, Lighthouse-pinned checkpoints) make the submission verifiable from the block explorers, which moves the project from "demo theater" to "production system the judges can audit live" — the single highest-signal credibility move in lablab's recent pattern of winning entries. Build clean, ship early, keep the demo live, and let the integrated architecture do the talking.